Deploying Keycloak for HA with the Operator (v2)

26.7.0

Deploy Keycloak for multi-cluster HA (stateless) with the Keycloak Operator as a building block.

This guide is describing a feature which is currently in preview. Please provide your feedback by joining this discussion while we're continuing to work on this.

This guide describes advanced Keycloak configurations for Kubernetes which are load tested and will recover from single Pod failures or site failures.

These instructions are intended for use with the setup described in the Concepts for multi-cluster deployments (v2) guide.

While this blueprint uses Kubernetes, the concepts are not limited to Kubernetes.

Prerequisites

- Kubernetes clusters running. Each Kubernetes cluster should have its own internal load balancing to leverage the startup, liveness and readiness probes of the Keycloak Pods.
- A load balancer in front of the clusters that is probing the URL `/lb-check` of the Keycloak clusters.
- A highly available database deployed with synchronous replication across sites, reachable from all Keycloak nodes in all clusters. The following example assumes a PostgreSQL database. Any database supported by Keycloak will work.
- Understanding of a Basic Keycloak deployment of Keycloak with the Keycloak Operator.

The following setup assumes a TLS passthrough setup, though this is not limited to this configuration.

Procedure

Perform the following steps in each Kubernetes cluster:

1. Determine the sizing of the deployment using the Concepts for sizing CPU and memory resources guide. For this setup, you might see approximately twice the CPU usage and write IOPS on your database, though your results might vary based on your use case.
2. Install the Keycloak Operator as described in the Keycloak Operator Installation guide.
3. Secure the database connection by creating a `ConfigMap` with the database certificate bundle:

```
kubectl --namespace keycloak create configmap keycloak-db-rootcert --from-fil
```

4. Deploy the Keycloak CR with the following values, using the resource requests and limits calculated in the first step:

```
apiVersion: k8s.keycloak.org/v2beta1
kind: Keycloak
```

```
metadata:
  labels:
    app: keycloak
  name: keycloak
  namespace: keycloak
spec:
  hostname:
    hostname: <KEYCLOAK_URL_HERE>
  resources:
    requests:
      cpu: "2"
      memory: "1250M"
    limits:
      cpu: "6"
      memory: "2250M"
  db:
    vendor: postgres
    url: jdbc:postgresql://<DATABASE_URL_HERE>:5432/keycloak 1
    poolMinSize: 30 2
    poolInitialSize: 30
    poolMaxSize: 30
    usernameSecret:
      name: keycloak-db-secret
      key: username
    passwordSecret:
      name: keycloak-db-secret
      key: password
  image: <KEYCLOAK_IMAGE_HERE> 3
  startOptimized: false 3
  update:
    strategy: Auto
  features:
    enabled:
      - stateless 4
  additionalOptions:
    - name: spi-cache-embedded--default--cluster-name
      value: <CLUSTER_NAME_HERE> 5
    - name: log-console-output
      value: json
    - name: metrics-enabled 6
      value: 'true'
    - name: event-metrics-user-enabled
      value: 'true'
    - name: db-tls-mode 7
      value: verify-server
  http:
    tlsSecret: keycloak-tls-secret
  instances: 3
  truststores:
    db:
      configMap:
        name: keycloak-db-rootcert 8
```

1 Set the database URL.

2 The database connection pool initial, max and min size should be identical to allow statement caching for the database. Adjust this number to meet the needs of your system. With the `stateless` feature, authentication sessions and action tokens are stored in the database, which increases database load compared to setups with an external Infinispan cluster. Adjust the pool size accordingly. See the Concepts for database connection pools guide for details.

3 Specify the URL to your custom Keycloak image. If your image is optimized, set the `startOptimized` flag to `true`.

4 Enable the `stateless` feature to remove the requirement for an external Infinispan cluster.

5 Set a unique cluster name that identifies this cluster. Each cluster must have a distinct name; no two clusters may share the same name. This option is only available if the `stateless` feature is enabled.

6 To be able to analyze the system under load, enable the metrics endpoint.

7 Secure the database connection.

8 Specify the `ConfigMap` name that contains the database certificate bundle. The Operator will automatically mount the file in the directory

`/opt/keycloak/conf/truststores/configmap-<config map name>/<file-name>`.

Verify the deployment

Confirm that the Keycloak deployment is ready.

```
kubectl wait --for=condition=Ready keycloaks.k8s.keycloak.org/keycloak
kubectl wait --for=condition=RollingUpdate=False keycloaks.k8s.keycloak.org/keycl
```

Afterward, verify that the external load balancer routes traffic to both clusters.

Optional: Disable sticky sessions

When running on OpenShift and the default passthrough Ingress setup as provided by the Keycloak Operator, the load balancing done by HAProxy is done by using sticky sessions based on the IP address of the source. When running load tests, or when having a reverse proxy in front of HAProxy, you might want to disable this setup to avoid receiving all requests on a single Keycloak Pod.

Add the following supplementary configuration under the `spec` in the Keycloak Custom Resource to disable sticky sessions.

```
spec:
  ingress:
    enabled: true
    annotations:
      # When running load tests, disable sticky sessions on the OpenShift HAProxy
      # to avoid receiving all requests on a single Keycloak Pod.
      haproxy.router.openshift.io/balance: roundrobin
      haproxy.router.openshift.io/disable_cookies: 'true'
```